

7교시: PostgreSQL 16/18 컨테이너 병렬 실행

수업 목표

- postgres:16과 postgres:18을 각각 다른 host port로 실행한다.
- 같은 container port 5432를 서로 다른 host port에 publish하는 의미를 설명한다.
- psql 또는 container 내부 psql로 각 DB version을 확인한다.

강의 전개

이 교시는 Day 1에서 가장 중요한 실습이다. 같은 종류의 service를 서로 다른 version으로 동시에 띄우고, host port만 다르게 연결하면 둘 다 접근할 수 있음을 직접 확인한다. 학생은 여기서 Docker의 장점인 version isolation, named container, named volume, port publishing을 한 번에 경험한다.

먼저 container 이름과 volume 이름을 분리한다. paperclip-pg16, paperclip-pg18은 container lifecycle을 구분하기 위한 이름이고, paperclip-pg16-data, paperclip-pg18-data는 data lifecycle을 구분하기 위한 volume이다. 이름을 대충 붙이면 cleanup 때 어떤 것이 어떤 version의 데이터인지 알 수 없게 된다.

그 다음 port mapping을 천천히 읽는다. PostgreSQL process는 container 내부에서 계속 5432를 사용한다. host에서는 PostgreSQL 16을 localhost:15432, PostgreSQL 18을 localhost:15433으로 접근한다. 이때 15432:5432의 왼쪽은 host port, 오른쪽은 container port다. 이 한 줄을 거꾸로 이해하면 이후 모든 DB 접속 문제가 꼬인다.

version 확인은 실행 성공보다 강한 확인 지점이다. container가 running 상태여도 DB가 아직 초기화 중이면 query가 실패할 수 있다. 그래서 docker logs에서 readiness message를 보고, psql 또는 docker exec로 SELECT version();을 실행한다. 최종 확인 지점은 "container가 있다"가 아니라 "15432는 16 계열, 15433은 18 계열을 반환했다"가 되어야 한다.

마지막에는 실패 경로를 같은 언어로 분류한다. pull이 느린 문제, password 누락, port 충돌, readiness 대기 부족, volume path 혼동, local psql client 부재는 서로 다른 문제다. host에 psql이 없어도 container 내부 psql로 확인할 수 있게 해 두면 실습 참여율이 높아진다.

실습 전 정리

같은 이름의 container가 남아 있으면 먼저 정리한다.

```
docker ps -a --filter name=paperclip-pg
```

이미 남아 있는 실습 container가 있고 삭제해도 되는 상태라면:

```
docker stop paperclip-pg16 paperclip-pg18
docker rm paperclip-pg16 paperclip-pg18
```

없다는 error가 나올 수 있다. 이 경우는 정리 대상이 없다는 뜻이다.

Hands-on 1: PostgreSQL 16 실행

```
docker run -d \
  --name paperclip-pg16 \
  -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=paperclip \
  -p 15432:5432 \
  -v paperclip-pg16-data:/var/lib/postgresql/data \
  postgres:16
```

확인한다.

```
docker ps --filter name=paperclip-pg16
docker logs paperclip-pg16
```

정상 기준:

- docker ps에서 0.0.0.0:15432->5432/tcp 또는 [::]:15432->5432/tcp가 보인다.
- logs에 database system is ready to accept connections 계열 메시지가 보인다.

Hands-on 2: PostgreSQL 18 실행

```
docker run -d \
  --name paperclip-pg18 \
  -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=paperclip \
  -p 15433:5432 \
  -v paperclip-pg18-data:/var/lib/postgresql \
  postgres:18
```

PostgreSQL 18 공식 image는 PGDATA 기본 경로가 version specific 경로로 바뀌었고, volume target도 /var/lib/postgresql 기준으로 안내된다. 그래서 16과 18의 volume을 절대 공유하지 않는다.

확인한다.

```
docker ps --filter name=paperclip-pg18
docker logs paperclip-pg18
```

정상 기준:

- docker ps에서 15433->5432/tcp가 보인다.
- postgres:18 image로 실행 중이다.

Hands-on 3: 각 version 접속 확인

host에 psql client가 있으면 아래 명령을 사용한다.

```
PGPASSWORD=postgres psql -h localhost -p 15432 -U postgres -d paperclip -c
"SELECT version();"
PGPASSWORD=postgres psql -h localhost -p 15433 -U postgres -d paperclip -c
"SELECT version();"
```

host에 psql이 없으면 container 안의 psql을 사용한다.

```
docker exec paperclip-pg16 psql -U postgres -d paperclip -c "SELECT
version();"
docker exec paperclip-pg18 psql -U postgres -d paperclip -c "SELECT
version();"
```

결과 판정

접속	기대 결과
localhost:15432	PostgreSQL 16 계열 version 출력
localhost:15433	PostgreSQL 18 계열 version 출력
둘 다 성공	같은 container port라도 host port가 다르면 병렬 실행 가능
하나만 실패	해당 container log, port mapping, password, readiness 확인

PostgreSQL 16/18 container 주의점

- 두 container가 모두 내부 port 5432를 써도 된다. 서로 다른 container namespace 안에서 실행되기 때문이다.
- host에서 동시에 접근하려면 host port는 달라야 한다. 예: 15432:5432, 15433:5432.
- 접속 실패가 password 문제인지, port mapping 문제인지, readiness 문제인지 분리해서 본다. 먼저 docker ps의 PORTS와 docker logs를 확인한다.

- 같은 container name을 재사용하면 실행이 실패한다. 이전 실습 container가 남아 있으면 삭제하거나 다른 name을 사용한다.

흔한 오해

오해	바로잡기
두 DB 모두 PostgreSQL이라 동시에 못 띄운다.	container 이름, volume, host port를 다르게 하면 동시에 실행할 수 있다.
container port도 바뀌어야 한다.	내부 PostgreSQL은 계속 5432를 쓴다. host port만 15432, 15433으로 다르게 publish한다.
같은 password를 쓰면 같은 DB다.	container, volume, port가 다르면 별도 DB instance다.
16과 18 volume을 공유해도 된다.	major version과 PGDATA 차이 때문에 공유하지 않는다.

공식 근거 링크

- PostgreSQL official image README: <https://github.com/docker-library/docs/blob/master/postgres/README.md>
- Docker Docs: Publishing and exposing ports, <https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/>
- Docker Docs: docker container logs, <https://docs.docker.com/reference/cli/docker/container/logs/>

다음 연결

다음 교시는 일부러 port 충돌을 만들고, 왜 같은 host port를 두 container가 동시에 사용할 수 없는지 확인한 뒤 정리와 주의할 점 확인으로 마감한다.